Structures and Unions

Arrays allow to define type of variables that can hold several data items of the same kind

- structure is another user defined data type that allows to combine data items of different kinds
- Structures are used to represent a record
- Suppose you want to keep track of your books in a library.
- The following attributes about each book can be stored

- Title
- Author
- Subject
- Book ID

Defining a Structure

To define a structure, the struct statement with *struct* keyword is used

- The struct statement defines a new data type, with more than one member.
- The format of the struct statement is as follows −
- struct [structure tag] {

- member definition;
- member definition;
- ...
- member definition;
- } [one or more structure variables];
- The structure tag is optional and each member definition is a normal variable definition, such as int i; or float f; or any other valid variable definition
- At the end of the structure's definition, before the final semicolon, you can specify one or more structure variables but it is optional.

Example to define Book structure

```
struct Books {
   char  title[50];
   char  author[50];
   char  subject[100];
   int   book_id;
} book;
```

Example to define a student structure

```
struct students {
    char  name[50];
    int       age;
    int  RollNo;
    char  course[10];
};
```

Accessing structure members

- To access any member of a structure, we use the member access operator (.)
- The member access operator is coded as a period between the structure variable name and the structure member that we wish to access
- You would use the keyword struct to define variables of structure type
- The following example shows how to use a structure in a program –

Example to show how to use a structure in a program

```
#include <stdio.h>
#include <string.h>

struct Books {
   char  title[50];
   char  author[50];
   char  subject[100];
   int   book_id;
};
```

Example contd..

```
int main( ) {

   struct Books Book1;        /* Declare Book1 of type Books */
   struct Books Book2;        /* Declare Book2 of type Books */

   /* book 1 specification */
   strcpy( Book1.title, "C Programming");
   strcpy( Book1.author, "Nuha Ali");
   strcpy( Book1.subject, "C Programming Tutorial");
   Book1.book_id = 6495407;

   /* book 2 specification */
   strcpy( Book2.title, "Telecom Billing");
   strcpy( Book2.author, "Zara Ali");
   strcpy( Book2.subject, "Telecom Billing Tutorial");
   Book2.book_id = 6495700;
```

Example contd..

```
/* print Book1 info */
   printf( "Book 1 title : %s\n", Book1.title);
   printf( "Book 1 author : %s\n", Book1.author);
   printf( "Book 1 subject : %s\n", Book1.subject);
   printf( "Book 1 book_id : %d\n", Book1.book_id);

   /* print Book2 info */
   printf( "Book 2 title : %s\n", Book2.title);
   printf( "Book 2 author : %s\n", Book2.author);
   printf( "Book 2 subject : %s\n", Book2.subject);
   printf( "Book 2 book_id : %d\n", Book2.book_id);
   return 0; }
```

Output of Example

- When the above code is compiled and executed, it produces the following result –

- Book 1 title : C Programming
- Book 1 author : Nuha Ali
- Book 1 subject : C Programming Tutorial
- Book 1 book_id : 6495407
- Book 2 title : Telecom Billing
- Book 2 author : Zara Ali
- Book 2 subject : Telecom Billing Tutorial
- Book 2 book_id : 6495700

Example 2: program to store and access “id, name and percentage” for one student

```
#include <stdio.h>
#include <string.h>
struct student
{
        int id;
        char name[20];
        float percentage;
};
```

Example 2 contd..

```
int main() {
        struct student record = {0}; //Initializing to null
        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;
        printf(" Id is: %d \n", record.id);
        printf(" Name is: %s \n", record.name);
        printf(" Percentage is: %f \n", record.percentage);
        return 0;
}
```

OUTPUT:

Id is: 1

Name is: Raju

Percentage is: 86.5

Difference between a variable, array and a structure

- A variable can store only one value
- An array can store a number of variables of the same type
- A structure can store a number of variables of different type

- each element in a C structure is called member

- It is a good practice to initialize a structure to null while declaring, if we don’t assign any values to structure members

Uses of structures

- Structures can be used to store huge data
- C Structures can be used to send data to the printer
- C Structures can interact with keyboard and mouse to store the data
- C Structures can be used to clear output screen contents

Example 3: Array of structures

```
#include <stdio.h>
#include <string.h>

struct student
{
    int id;
    char name[30];
    float percentage;
};
```

Example 3 contd..

```
int main()
{
    int i;
    struct student record[2];

    /* 1st student's record*/
    record[0].id=1;
    strcpy(record[0].name, "Raju");
    record[0].percentage = 86.5;

    /*2nd student's record  */
    record[1].id=2;
    strcpy(record[1].name, "Surinder");
    record[1].percentage = 90.5;
```

Example 3 contd..

```
/* 3rd student's record */
    record[2].id=3;
    strcpy(record[2].name, "Tyagi");
    record[2].percentage = 81.5;
    for(i=0; i<3; i++)
    {
        printf("     Records of STUDENT : %d \n", i+1);
        printf(" Id is: %d \n", record[i].id);
        printf(" Name is: %s \n", record[i].name);
        printf(" Percentage is: %f\n\n",record[i].percentage);
    }
    return 0;
}
```

Output of Example 3

- OUTPUT:
- Records of STUDENT : 1
- Id is: 1
- Name is: Raju
- Percentage is: 86.500000
- Records of STUDENT : 2
- Id is: 2
- Name is: Surendren
- Percentage is: 90.500000
- Records of STUDENT : 3
- Id is: 3
- Name is: Thiyagu
- Percentage is: 81.500000

Example 4: Program for declaring many structure variables

- In this program, two structure variables “record1” and “record2” are declared for same structure and different values are assigned for both structure variables
- Separate memory is allocated for both structure variables to store the data
- #include <stdio.h>
- #include <string.h>
-
- struct student
- {
- int id;
- char name[30];
- float percentage;
- };
-

Example 4 contd..

```
int main()
{
    int i;
    struct student record1 = {1, "Raju", 90.5};
    struct student record2 = {2, "Mani", 93.5};

    printf("Records of STUDENT1: \n");
    printf("  Id is: %d \n", record1.id);
    printf("  Name is: %s \n", record1.name);
    printf("  Percentage is: %f \n\n", record1.percentage);

    printf("Records of STUDENT2: \n");
    printf("  Id is: %d \n", record2.id);
    printf("  Name is: %s \n", record2.name);
    printf("  Percentage is: %f \n\n", record2.percentage);

    return 0;
}
```

Output of Example 4

OUTPUT:

- Records of STUDENT1:
- Id is: 1
- Name is: Raju
- Percentage is: 90.500000
- Records of STUDENT2:
- Id is: 2
- Name is: Mani
- Percentage is: 93.500000

Passing struct to function

- A structure can be passed to any function from main function or from any sub function
- Structure definition will be available within the function only.
- It won’t be available to other functions unless it is passed to those functions by value or by address(reference).
- Otherwise, we have to declare structure variable as global variable

- That means, structure variable should be declared outside the main function
- So, this structure will be visible to all the functions in a C program

Example program to pass structure to function by value

- In this program, the whole structure is passed to another function by value
- It means the whole structure is passed to another function with all members and their values
- So, this structure can be accessed from called function
- This concept is very useful while writing very big programs in C

```
#include <stdio.h>
#include <string.h>

struct student
{
        int id;
        char name[20];
        float percentage;
};

void func(struct student record);
```

Example program to pass structure to function by value contd..

```
int main()
{
        struct student record;
        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;
        func(record);
        return 0;
}
void func(struct student record)
{
        printf(" Id is: %d \n", record.id);
        printf(" Name is: %s \n", record.name);
        printf(" Percentage is: %f \n", record.percentage);
}
```

Example program to pass structure to function by value contd..

- OUTPUT:
- Id is: 1
- Name is: Raju
- Percentage is: 86.500000

Example: Passing structure to a function by reference

```
#include <stdio.h>
#include <string.h>

struct student
{
        int id;
        char name[20];
        float percentage;
};
```

Example: Passing structure to a function by reference contd..

```
void func(struct student *record);

int main()
{
        struct student record;

        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;
        func(&record);
        return 0;
}
```

Example: Passing structure to a function by reference contd..

```
void func(struct student *record)
{
        printf(" Id is: %d \n", record->id);
        printf(" Name is: %s \n", record->name);
        printf(" Percentage is: %f \n", record->percentage);
}
```

OUTPUT:

- Id is: 1
- Name is: Raju
- Percentage is: 86.500000

Example program to declare a structure variable as global

- Structure variables also can be declared as global variables as other variables in C
- So, When a structure variable is declared as global, then it is visible to all the functions in a program
- Hence there is no need to pass the structure to any function separately
- #include <stdio.h>
- #include <string.h>
- struct student

  {
- int id;
- char name[20];
- float percentage;
- };
- struct student record; // Global declaration of structure

Example program to declare a structure variable as global contd..

```
void structure_demo();

int main()
{
        record.id=1;
        strcpy(record.name, "Raju");
        record.percentage = 86.5;

        structure_demo();
        return 0;
}
```

Example program to declare a structure variable as global contd..

```
void structure_demo()
{
        printf(" Id is: %d \n", record.id);
        printf(" Name is: %s \n", record.name);
        printf(" Percentage is: %f \n", record.percentage);
}
```

OUTPUT:

Id is: 1

Name is: Raju

Percentage is: 86.500000

Example: Function returning a structure

- #include <stdio.h>
- #include <string.h>
- struct student {
- char firstname[64];
- char lastname[64];
- char id[64];
- int score;
- };
- Let us create a function that will return variable of type student structure

Example: Function returning a structure contd..

```
struct student getDetail(void);
void displayDetail(struct student std);
int main() {
/*creating a student structure array variable*/
  struct student stdArr[3];
  /* other variables*/
  int i;
  // taking user input
  for (i = 0; i < 3; i++) {
    printf("Enter detail of student #%d\n", (i+1));
    stdArr[i] = getDetail();
  }
```

Example: Function returning a structure contd..

```
for (i = 0; i < 3; i++) {
    printf("\nStudent #%d Detail:\n", (i+1));

displayDetail(stdArr[i]);
  }
  return 0;
}

struct student getDetail(void) {
  /*temp structure variable*/
  struct student std;
```

Example: Function returning a structure contd..

```
printf("Enter First Name: ");
 scanf("%s", std.firstname);
printf("Enter Last Name: ");
 scanf("%s", std.lastname);
 printf("Enter ID: ");
 scanf("%s", std.id);
 printf("Enter Score: ");
 scanf("%d", &std.score);

 return std;
}
```

Example: Function returning a structure contd..

```
void displayDetail(struct student std) {
  printf("Firstname: %s\n", std.firstname);
  printf("Lastname: %s\n", std.lastname);
  printf("ID: %s\n", std.id);
  printf("Score: %d\n", std.score);
}
```

Output:

Output:

Output contd..

- Enter detail of student #1
- Enter First Name: Bruce
- Enter Last Name: Wayne
- Enter ID: dc-01
- Enter Score: 8
- Enter detail of student #2
- Enter First Name: Peter
- Enter Last Name: Parker
- Enter ID: mc-01
- Enter Score: 9
- Enter detail of student #3
- Enter First Name: Tony
- Enter Last Name: Stark
- Enter ID: mc-02
- Enter Score: 7

Output contd..

- Student #1 Detail:
- Firstname: Bruce
- Lastname: Wayne
- ID: dc-01
- Score: 8

- Student #2 Detail:
- Firstname: Peter
- Lastname: Parker

Output contd..

- ID: mc-01
- Score: 9

- Student #3 Detail:
- Firstname: Tony
- Lastname: Stark
- ID: mc-02
- Score: 7

Pointers and Structures

- C structure can be accessed in two ways in a C program
- Using normal structure variable
- Using pointer variable
- Dot(.) operator is used to access the data using normal structure variable
- arrow (->) is used to access the data using pointer variable
- Let us see how to access structure data using pointer variable in a C program.

Example program for a structure using pointer

- In this program, “record1” is normal structure variable and “ptr” is pointer structure variable
- Dot(.) operator is used to access the data using normal structure variable and arrow(->) is used to access data using pointer variable
- #include <stdio.h>
- #include <string.h>
-
- struct student
- {
- int id;
- char name[30];
- float percentage;
- };

Example program for a structure using pointer contd..

```
int main()
{
    struct student record1 = {1, "Raju", 90.5};
    struct student *ptr;

    ptr = &record1;

       printf("Records of STUDENT1: \n");
       printf("  Id is: %d \n", ptr->id);
       printf("  Name is: %s \n", ptr->name);
       printf("  Percentage is: %f \n\n", ptr->percentage);

    return 0;
}
```

Example program for a structure using pointer contd..

- OUTPUT:
- Records of STUDENT1:
- Id is: 1
- Name is: Raju
- Percentage is: 90.500000

Nested Structure

- Nested structure in C is a structure within structure
- One structure can be declared inside other structure as we declare structure members inside a structure
- The structure variables can be a normal structure variable or a pointer variable to access the data
- Structure within structure in C using normal variable
- Structure within structure in C using pointer variable

Example program for Structure within Structure using Normal Variable

- This program explains how to use structure within structure in C using normal variable
- “student_college_detail’ structure is declared inside “student_detail” structure in this program- Both structure variables are normal structure variables.
- Members of “student_college_detail” structure are accessed by 2 dot(.) operator and members of “student_detail” structure are accessed by single dot(.) operator
- #include <stdio.h>
- #include <string.h>
-
- struct student_college_detail
- {
- int college_id;
- char college_name[50];
- };

Example program for Structure within Structure contd..

```
struct student_detail
{
    int id;
    char name[20];
    float percentage;
    /*structure within structure*/
    struct student_college_detail clg_data;
}stu_data;
```

Example program for Structure within Structure contd..

```
int main()
{
    struct student_detail stu_data = {1, "Raju", 90.5, 71145,
                                      "Anna University"};
    printf(" Id is: %d \n", stu_data.id);
    printf(" Name is: %s \n", stu_data.name);
    printf(" Percentage is: %f \n\n", stu_data.percentage);

    printf(" College Id is: %d \n",
                stu_data.clg_data.college_id);
    printf(" College Name is: %s \n",
                stu_data.clg_data.college_name);
    return 0;
}
```

Example program for Structure within Structure contd..

- OUTPUT:
- Id is: 1
- Name is: Raju
- Percentage is: 90.500000
- College Id is: 71145
- College Name is: Anna University

Example program for Structure within Structure using pointer variable

- This program explains how to use structure within structure in C using pointer variable
- “student_college_detail’ structure is declared inside “student_detail” structure
- one normal structure variable and one pointer structure variable is used in this program.
- Please note that combination of .(dot) and ->(arrow) operators are used to access the structure member which is declared inside the structure
- #include <stdio.h>
- #include <string.h>
- struct student_college_detail
- {
- int college_id;
- char college_name[50];
- };

Example program for Structure within Structure using pointer variable contd..

```
struct student_detail
{
    int id;
    char name[20];
    float percentage;
    /* structure within structure*/
    struct student_college_detail clg_data;
}stu_data, *stu_data_ptr;
```

Example program for Structure within Structure using pointer variable contd..

```
int main()
{
  struct student_detail stu_data = {1, "Raju", 90.5, 71145,
                                    "Anna University"};
    stu_data_ptr = &stu_data;

    printf(" Id is: %d \n", stu_data_ptr->id);
    printf(" Name is: %s \n", stu_data_ptr->name);
    printf(" Percentage is: %f \n\n",
                        stu_data_ptr->percentage);
```

Example program for Structure within Structure using pointer variable contd..

```
printf(" College Id is: %d \n",
                    stu_data_ptr->clg_data.college_id);
   printf(" College Name is: %s \n",
                    stu_data_ptr->clg_data.college_name);
   return 0;
}
```

OUTPUT:

Id is: 1

Name is: Raju

Percentage is: 90.500000

College Id is: 71145

College Name is: Anna University

Union in C

- C Union is also like structure, i.e. collection of different data types which are grouped together
- Each element in a union is called member.
- Union and structure in C are same in concepts, except allocating memory for their members.
- Structure allocates storage space for all its members separately.
- Whereas, Union allocates one common storage space for all its members

Structure versus union

- We can access only one member of union at a time
- We can’t access all member values at the same time in union
- But, structure can access all member values at the same time
- This is because, Union allocates one common storage space for all its members
- Where as Structure allocates storage space for all its members separately.
- Many union variables can be created in a program and memory will be allocated for each union variable separately.

Syntax of union definition

- union tag_name
- {
- data type var_name1;
- data type var_name2;
- data type var_name3;
- };
- Example:
- union student
- {
- int mark;
- char name[10];
- float average;
- };

Declaring union variables

- Declaring union using normal variable:
- union student report;

- Declaring union using pointer variable:
- union student *report, rep;

Example: Using union

```
#include <stdio.h>
#include <string.h>
union student
{
        char name[20];
        char subject[20];
        float percentage;
};
int main()
{
   union student record1;
   union student record2;
   /* assigning values to record1 union variable*/
     strcpy(record1.name, "Raju");
     strcpy(record1.subject, "Maths");
     record1.percentage = 86.50;
```

Example: Using union contd..

```
printf("Union record1 values example\n");
      printf(" Name      : %s \n", record1.name);
      printf(" Subject   : %s \n", record1.subject);
      printf(" Percentage : %f \n\n", record1.percentage);

   // assigning values to record2 union variable
      printf("Union record2 values example\n");
      strcpy(record2.name, "Mani");
      printf(" Name      : %s \n", record2.name);

      strcpy(record2.subject, "Physics");
      printf(" Subject   : %s \n", record2.subject);

      record2.percentage = 99.50;
      printf(" Percentage : %f \n", record2.percentage);
      return 0;
}
```

Output of example: Using union

- Union record1 values example
- Name :
- Subject :
- Percentage : 86.500000;
- Union record2 values example
- Name : Mani
- Subject : Physics
- Percentage : 99.500000

Explanation of example: Using union

- There are 2 union variables declared in this program to understand the difference in accessing values of union members.
- Record1 union variable:
- “Raju” is assigned to union member “record1.name” . The memory location name is “record1.name” and the value stored in this location is “Raju”.
- Then, “Maths” is assigned to union member “record1.subject”. Now, memory location name is changed to “record1.subject” with the value “Maths” (Union can hold only one member at a time).
- Then, “86.50” is assigned to union member “record1.percentage”. Now, memory location name is changed to “record1.percentage” with value “86.50”.
- Like this, name and value of union member is replaced every time on the common storage space.

Explanation of example: union program contd..

- So, we can always access only one union member for which value is assigned at last. We can’t access other member values.
- So, only “record1.percentage” value is displayed in output. “record1.name” and “record1.percentage” are empty.
- Record2 union variable:
- If we want to access all member values using union, we have to access the member before assigning values to other members as shown in record2 union variable in this program.
- Each union members are accessed in record2 example immediately after assigning values to them.
- If we don’t access them before assigning values to other member, member name and value will be over written by other member as all members are using same memory.
- We can’t access all members in union at same time but structure can do that.

C Structure	C Union
Structure allocates storage space for all its members separately.	Union allocates one common storage space for all its members. Union finds that which of its member needs high storage space over other members and allocates that much space
Structure occupies higher memory space.	Union occupies lower memory space over structure.
We can access all members of structure at a time.	We can access only one member of union at a time.

structure example:
struct student
{
int mark;
char name[6];
double average;
};
	Union example:
	union student
	{
	int mark;
	char name[6];
	double average;
	};

Uses of union

- Union allows access to a common location by different data types, for example hardware input/output access, bitfield and word sharing, or type punning. Unions can also provide low-level polymorphism.
- Polymorphism is the ability of an object to take on many forms.
- Type punning is a common term for any programming technique that subverts or circumvents or disrupts the type system of a programming language
- Type punning is a form of pointer aliasing where two pointers refer to the same location in memory but represent that location as different types
- Aliasing refers to the situation where the same memory location can be accessed using different names.
- Unions provide an efficient way of using the same memory location for multiple-purpose.

Pointer to Union in C

- Pointer to union can be created just like other pointers to primitive data types.
- Consider the below union declaration:
- union number{
- int a;
- int b;
- };
- Here, a and b are the members of the union number.

- Union variable declaration:

- union number n;
- Here, n is the variable of number which is of type union.

Pointer to union declaration and initialization

union number *ptr = &n;

Here, ptr is the pointer to the union and it is assigning with the address of the union variable n.

- Accessing union member using pointer to union:

- Syntax:
- pointer_name->member;

- Example:
- // to access members a and b using ptr
- ptr->a;
- ptr->b;

Example: C program to demonstrate pointer to union in C

```
#include <stdio.h>

int main()
{
    /* union declaration*/
    union number {
        int a;
        int b;
    };

    /* union variable declaration*/
    union number n = { 10 }; /* a will be assigned with 10*/
```

Example: C program to demonstrate pointer to union contd..

```
/* pointer to union*/
    union number* ptr = &n;
    printf("a = %d\n", ptr->a);
    /*changing the value of a*/
    ptr->a = 20;
    printf("a = %d\n", ptr->a);
    // changing the value of b
    ptr->b = 30;
   printf("b = %d\n", ptr->b);
    return 0;
}
```

Output:

- a = 10
- a = 20
- b = 30

Union of Structures

- As structure can be nested within another, there can be union in a union, union in a structure, or a structure in a union
- Let us see an example of structures nested in a union

```
#include<stdio.h>
int main( )
{
 struct a
{
   int i;
    char c[2];
   };
   struct b
  {
    int j;
    char d[2];
};
  union z
  {
     struct a key;
      struct b data;
   } strange;
```

Example: Program with structures in a union

```
strange.key.i = 512;
   strange.data.d[0] = 0;
    strange.data.d[1] = 32;
    printf("%d\n",strange.key.i);
      printf("%d\n",strange.data.j);
      printf("%d\n",strange.key.c[0]);
      printf("%d\n",strange.data.d[0]);
       printf("%d\n",strange.key.c[1]);
       printf("%d\n",strange.data.d[1]);
      return 0;
  }
```

Output:
```
512
512
  0
  0
 32
 32
```